Stochastic Simulation Assignment 2

Assignment C: Limit Order Book with Market Impact

Petr Goncharov

May 2026

1 Problem and Model Overview

In this Assignment it is required to simulate a limit order book. Initial mid-price starts at $m_0 = 100$ and evolves through series of trades where resting limit orders are matched one to one at the best opposite price when market orders arrive. Additionally limit orders can be canceled after a random patience waiting time. The simulation is written in Python and uses provided des_library package.

Parameters that were used are assignment defaults: $\lambda_0 = 5$, $\lambda_{\min} = 0.5$, $\alpha = 0.05$, $q_L = 0.7$, $\bar{v} = 10$, $\bar{\delta} = 0.5$, $\bar{\gamma} = 60$, $m^\star = 100$, $\Delta_{\max} = 5$, $\kappa = 0.1$. And two impact factors that are compared: $\eta = 0.02$ and $\eta = 0.10$.

2 Event Structure

Two main event types were used on top of des_library — a single constantly recycled Arrival event class and a per order Cancellation_event.

Arrival event. Every arrival is independently a limit order with probability $q_L = 0.7$ or a market order with probability $1 - q_L = 0.3$. After processing, the next arrival is scheduled at $t + \mathrm{Exp}(1/\lambda(m_t))$ using the *current* mid price. Because the mid only changes at trades and trades occur only inside an arrival, sampling the next inter arrival immediately after handling the current order yields an exact realization of the state dependent Poisson process without an additional rate-update event or stale arrival cancellation.

Limit order case. With probability 1/2 a buy limit is placed at $m_t - \delta$, otherwise a sell limit at $m_t + \delta$, with $\delta \sim \mathrm{Exp}(\bar{\delta})$. Buy limits whose price is ≤ 0 are rejected. The resting order is appended to its side of the book and a dedicated `Cancellation_event` is scheduled at $t + \mathrm{Exp}(\bar{\gamma})$.

Market order case. With probability 1/2 the order is a buy, otherwise a sell. A market buy removes a single resting order at the best ask and a market sell removes one at the best bid. If the relevant side is empty the order is rejected. A successful match is a *trade*: the matched limit time is recorded and the mid price is updated.

Cancellation event. When a Cancellation_event fires, the book class attempts to remove the referenced order. If the order has already been matched, the cancellation is not executed. Otherwise the order is removed and counted as a canceled. Cancellations do not trigger a rate update.

3 Data Collection

Each statistic is updated only at events that can change its value.

- **Time-to-fill** (SampleStatistic): on each successful match, record $t - t_{\text{arrival}}$ for the removed limit order. Cancelled and unfilled orders are excluded.
- **Cancellation rate** (Counter): incremented inside the book's `cancel` method only when an order is actually removed; no-op cancellations are not counted.
- **Market-order rejection rate** (Counter): increments when market order can not be executed due to opposite side being empty.
- **Bid ask spread** (TimeWeightedStatistic): the time-weighted average of $ask\_t - bid\_t$.
- **Mid price volatility:** mid price is sampled every 1 second by a recurring Sample event which saves the standard deviation of the log-returns excluding intervals where m_t did not change or where $m_t \leq 0$.
- **Arrival rate series:** the value of $\lambda(m_t)$ is logged on every arrival.

4 Steady-State Methodology

Regenerative method. The natural regeneration state for this limit order book is an empty book with $m_t = m^\star = 100$. A detector on `Simulation.on_after_event` was implemented that counted all transitions into the state - $\{\text{book empty } \textit{and } |m_t - m^\star| < 0.5\}$. Over a 10^5-second run, no such transitions were found. Because with given default parameters it is almost impossible for system to reach this state after simulation start. Hence Batch method was picked as the primary estimator.

Batch method was used to estimate steady-state means and 95% confidence intervals. Simulation splits into K partitions(batches) of equal length after an initial warmup. Within each batch the metric increments are accumulated using the difference of the global collectors' running totals. The batch's mean is then pushed into a metric specific SampleStatistic. The point estimate and 95% CI for each metric are obtained from these K batch means.

Warmup. Warmup is performed at simulation initialization by discarding first $T_{\text{warm}} = 1000$ to push state from no liquidity and empty book to stationary regime. The value $T_{\text{warm}} = 1000$ was picked after inspecting the time series of the cumulative time-to-fill mean with different T_{warm} values.

Batch length. Batch with length $T_{\text{batch}} = 200$ was chosen for experiments. With patience mean $\bar{\gamma} = 60$ this is roughly three patience times, which is approximately enough time for successive batches to be uncorrelated.

Stopping rule. The run was extended until the relative half-width $h/|\hat{\mu}|$ of the primary metrics (time-to-fill and cancellation rate) was ≤ 0.10. The final configuration ran for 495 batches.

5 Verification

5.1 V1

With $\eta = 0$, $\alpha = 0$, cancellations disabled, single-side market-buy arrivals at rate $\lambda = 0.5$ and opposite-side liquidity at rate $\mu = 1.0$, the matching system reduces to an M/M/1 queue. The analytical time is $E[W] = 1/(\mu - \lambda) = 2$. Simulation produced $\hat{W} = 1.978$ with 95% CI $[1.871, 2.085]$ over 89 batches which supports correctness of the event structure.

5.2 V2

Keeping V1 assumptions but with deterministic completions every $1/\mu$ seconds. For $\lambda = 0.5$, $\mu = 1.0$ the formula gives $E[W_q] = \rho/(2\mu(1-\rho)) = 0.5$ and the full time is $E[W] = E[W_q] + 1/\mu = 1.5$. Computed simulation produced $\hat{W} = 1.509$ with 95% CI $[1.469, 1.548]$ over 89 batches which confirms deterministic service reduction.

6 Comparison Experiment: $\eta = 0.02$ vs. $\eta = 0.10$

Full model was computed under both impact levels with all other parameters equal. Results are in Table 1.

Metric	$\eta = 0.02$ (low)		$\eta = 0.10$ (high)	
	mean	95% CI	mean	95% CI
Time-to-fill (s)	21.14	[20.76, 21.52]	21.09	[20.74, 21.45]
Cancellation rate (/s)	1.472	[1.445, 1.499]	1.018	[1.000, 1.036]
Rejection rate (/s)	≈ 0	[0, 0]	≈ 0	$[-2{\cdot}10^{-5},\ 1{\cdot}10^{-4}]$
Time-avg spread	-0.625	$[-0.663,\ -0.587]$	-3.45	$[-3.52,\ -3.37]$
Mid-price volatility	0.0036	—	0.0164	—

Table 1: Estimates with 95% batch-means CIs for the two impact regimes.

Negative spread An important quantitative finding in these experiment is spread value being systematically negative. One possible explanation for this phenomenon is limitation of this model which does not permit limit orders to match against each other which produces a crossed book. Mechanically, the best bid is the max of all existing bids and the best ask the min of all existing asks hence whenever the mid drifts further than $\bar{\delta}$ within a patience window $\bar{\gamma}$, those extremes cross. Positive spread can be achieved by adding limit to limit order resolution.

Cancellation rate decreases from 1.472/s to 1.018/s as η grows because higher drift carries the mid price into regions of lower $\lambda(m_t)$ which suppresses arrivals and hence consecutively inserts.

Confidence-interval overlap. Of the four primary metrics, only time-to-fill shows overlapping CIs between $\eta = 0.02$ and $\eta = 0.10$, hence that metric does not have significant statistical difference under the these two regimes. Cancellation rate, and spread have disjoint CIs, so their differences are statistically significant at the 95% level. Rejection rate is zero with both η after warmup which is expected.

Arrival rate and mid price trajectory Figure 1 shows the realized arrival rate $\lambda(m_t)$ over time; Figure 2 the corresponding mid-price trajectories.

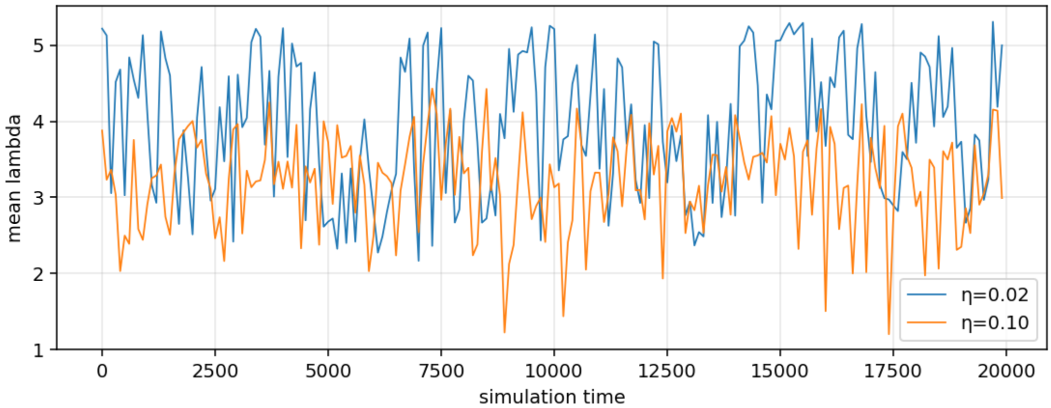

Figure 1: Realized state-dependent arrival rate $\lambda(m_t)$.

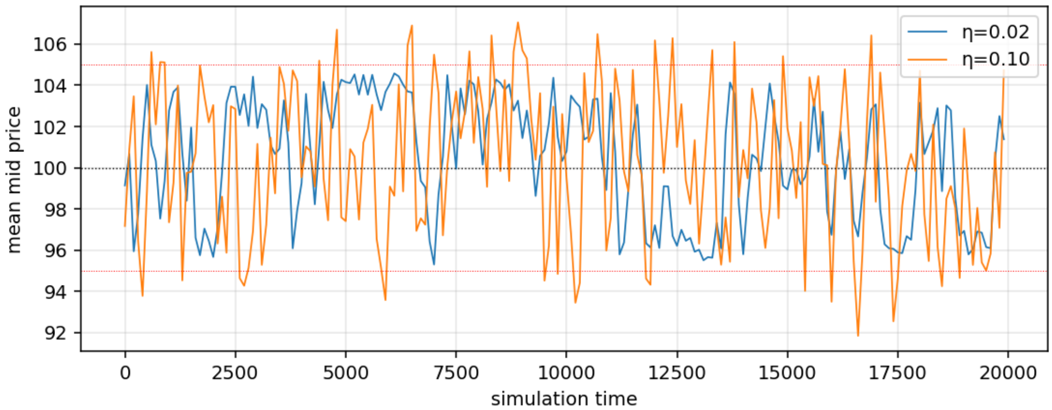

Figure 2: Mid price trajectories under low and high impact.

7 Extensions

It is possible to add limit-vs-limit auto-matching on insert, which will solve the issue with negative spread and bring the model closer to reality.